the node you were trying to remove. This thread won't access it anymore. If you're the last thread to hold a reference (because another thread removed it from the stack), the internal reference count will be 1, so subtracting 1 will set the count to zero. In this case, you can delete the node here before you loop ❽.

So far, you've been using the default `std::memory_order_seq_cst` memory ordering for all your atomic operations. On most systems these are more expensive in terms of execution time and synchronization overhead than the other memory orderings, and on some systems considerably so. Now that you have the logic of your data structure right, you can think about relaxing some of these memory-ordering requirements; you don't want to impose any unnecessary overhead on the users of the stack. So, before leaving your stack behind and moving on to the design of a lock-free queue, let's examine the stack operations and ask ourselves, can we use more relaxed memory orderings for some operations and still get the same level of safety?

7.2.5. Applying the memory model to the lock-free stack

Before you go about changing the memory orderings, you need to examine the operations and identify the required relationships between them. You can then go back and find the minimum memory orderings that provide these required relationships. In order to do this, you'll have to look at the situation from the point of view of threads in several different scenarios. The simplest possible scenario has to be where one thread pushes a data item onto the stack and another thread then pops that data item off the stack some time later, so we'll start from there.

In this simple case, three important pieces of data are involved. First is the `counted_node_ptr` used for transferring the data: `head`. Second is the `node` structure that `head` refers to, and third is the data item pointed to by that node.

The thread doing the `push()` first constructs the data item and the `node` and then sets `head`. The thread doing the `pop()` first loads the value of